



Grado en Ingeniería Informática

Estructuras de Datos Avanzadas

Examen práctico de implementación Convocatoria ordinaria 17 de enero de 2026

Tiempo máximo: 1 hora

Instrucciones:

- La duración del examen es de 60 minutos.
- La puntuación de cada ejercicio se indica junto al enunciado.
- No se responderán preguntas sobre los enunciados. La interpretación de estos forma parte del examen.
- No está permitido el uso de calculadoras, ni la consulta de libros, apuntes o dispositivos electrónicos de ningún tipo durante el examen.
- Desde el Aula Virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen y las pruebas unitarias asociadas.
- El estudiante debe generar un fichero ZIP con el directorio en el que esté el código desarrollado durante el examen y entregarlo en la actividad “Examen práctico” disponible en Aula Virtual.

Consideraciones adicionales:

- No se pueden añadir nuevas clases, métodos o propiedades públicas.
- Solo se pueden añadir nuevas clases si son privadas. A las clases ya dadas solo se les puede añadir métodos y propiedades con visibilidad privada.
- No se pueden modificar las cabeceras de los métodos ya dados.
- El código del proyecto proporcionado se ha probado utilizando **Oracle OpenJDK 24.0.2** (disponible en el Escritorio de Desarrollo de MyApps) y **JUnit 5.8.1**.
- Para poder evaluar la implementación de un método, es necesario que las pruebas unitarias asociadas a este se ejecuten correctamente, mostrando el comportamiento esperado.

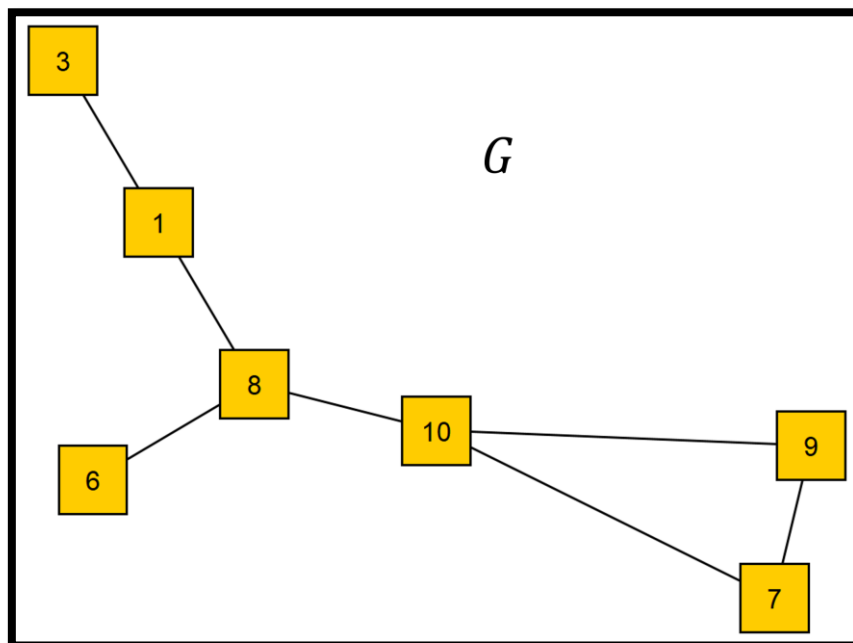
Ejercicio 1: GraphOperations

Asumir que todos los grafos son no dirigidos y no tienen bucles (una arista que conecta un vértice consigo mismo).

existeCaminoDeLongitudMenorOIgualAN (3 puntos)

Determina, dado un grafo, un vértice inicial, un vértice final, y un entero n , si existe un camino de longitud menor o igual a n del vértice inicial al vértice final.

Por ejemplo, supongamos que tenemos el siguiente grafo:

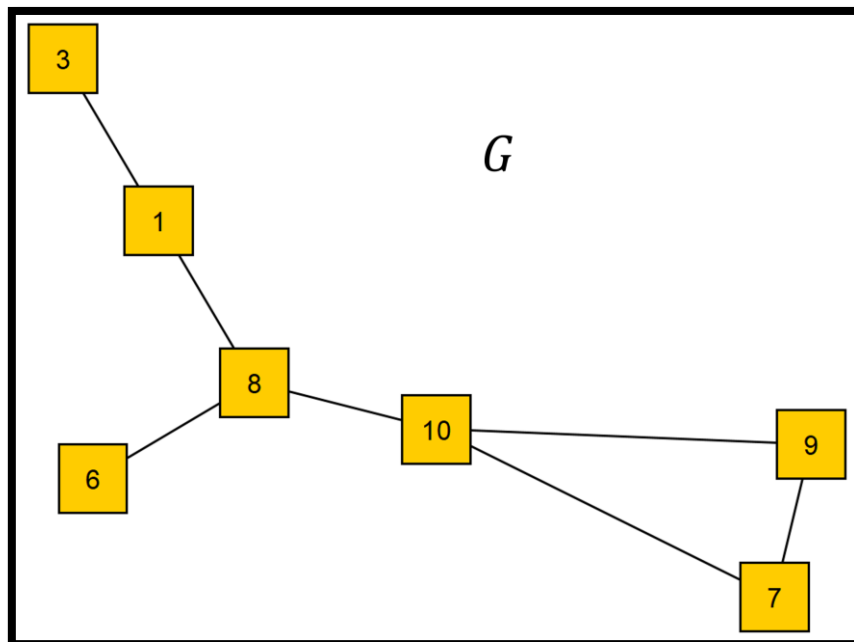


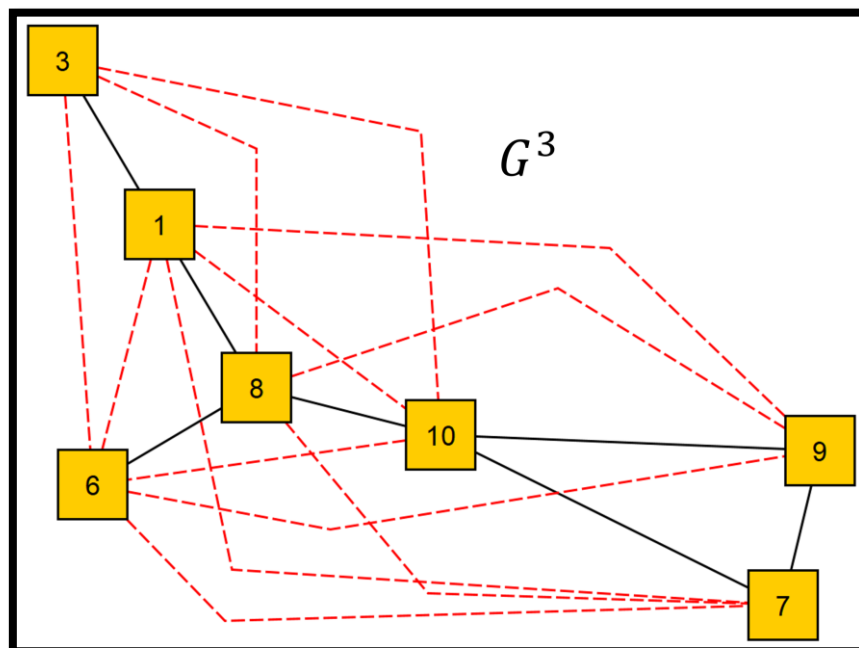
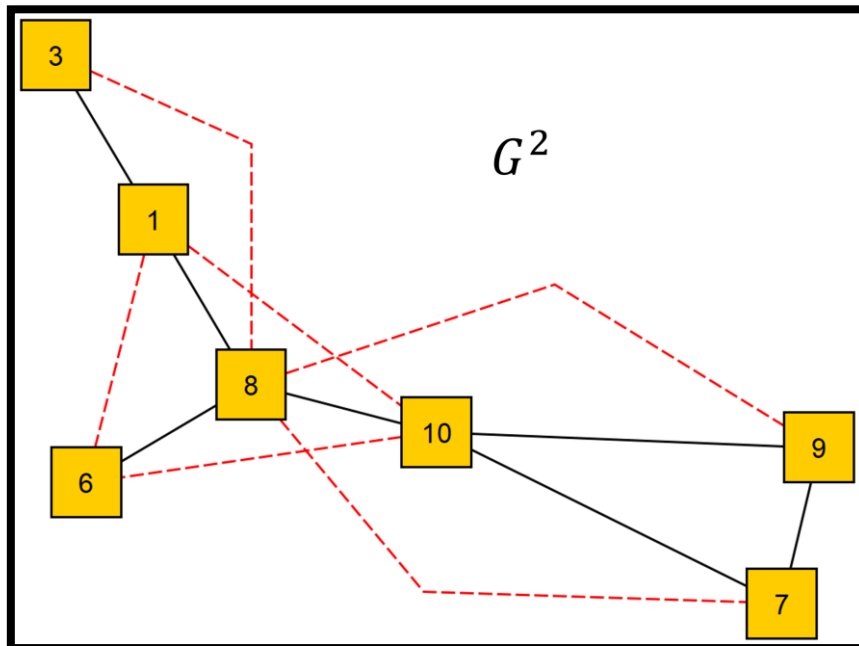
- Existe un camino de longitud menor o igual a 2 entre los vértices 8 y 3.
- Existe un camino de longitud menor o igual a 2 entre los vértices 8 y 9.
- No existe un camino de longitud menor o igual a 3 entre los vértices 9 y 3.
- Existe un camino de longitud menor o igual a 2 entre los vértices 10 y 9.
- No existe un camino de longitud menor o igual a 2 entre los vértices 6 y 3.
- Existe un camino de longitud menor o igual a 3 entre los vértices 6 y 3.

La respuesta se tiene que conseguir de manera eficiente. Lanzar cualquiera de los recorridos disponibles en la clase *GraphAlgorithms* supone una calificación de 0 en este ejercicio. No obstante, se puede (y se recomienda) resolver el ejercicio adaptando alguno de los métodos presentes en dicha clase.

kPower (3 puntos)

Devuelve la potencia k-ésima de un grafo. El grafo k-ésimo potencia de un grafo G es un grafo que contiene una arista entre dos vértices u y v si existe un camino de longitud menor o igual a k entre u y v en G . A continuación, se pueden observar tres ejemplos: G , G^2 y G^3 . G es el grafo original sobre el que se va a trabajar. G^2 contiene todas las aristas del grafo original y, además, una arista para cada par de nodos entre los que existe un camino de longitud menor o igual a 2 en el grafo original. Estas aristas están marcadas de color rojo con una línea discontinua. Del mismo modo, G^3 contiene todas las aristas del grafo original y, además, una arista para cada par de nodos entre los que existe un camino de longitud menor o igual a 3 en el grafo original.





Si el grafo es nulo, lanza la excepción `IllegalArgumentException`.

Si k es menor que 1, lanza la excepción `IllegalArgumentException`.

Para implementar este método, se puede hacer uso del método anterior, `existeCaminoDeLongitudMenorOIgualAN()`, al que se puede llamar repetidas veces.



Ejercicio 2: TreeOperations

cumplePropiedadesMonticulo (4 puntos)

Comprueba si un árbol binario dado cumple las propiedades de un montículo. Es decir, si el árbol es casi completo y el elemento que hay en cada nodo es más prioritario que los elementos que hay en los hijos de dicho nodo.

Si el árbol es nulo, lanza `IllegalArgumentException`.

Si el árbol está vacío, devuelve verdadero.

Ejercicio 3: ExtendedBreadthFirstTreeIterator

remove (2 puntos)

Implementa el método *remove()* del iterador *ExtendedBreadthFirstTreeIterator*. Este método elimina el último elemento devuelto por el iterador del árbol que se está recorriendo. Para ello, debe utilizar el método *remove()* del árbol por el que se está iterando. Por supuesto, el iterador debe seguir iterando por el árbol si quedan elementos por recorrer que no se han eliminado del árbol.

Para implementar este método, se pueden modificar los miembros privados de la clase *ExtendedBreadthFirstTreeIterator* y el código de los métodos. No obstante, no se pueden modificar las cabeceras de la clase ni de sus métodos.